Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Remote Procedure Calling (RPC)

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- What is RPC?
- How is RPC different to other architectural styles for building services?
- What are the conventional steps of RPC?
- Benefits and Downsides of RPC
- gRPC

UNIVERSITY of HULL

Method Invocation

Consider a C# method that takes an array of integers as its parameter and returns an integer equal to the sum of the integers:

```
public int Add(int[] integers)
{
    int toReturn = 0;
    foreach(int i in integers)
    {
        toReturn += i;
    }
    return toReturn;
}
```

When we call this method locally from code, we are **invoking** it:

```
Add(new int[] {1,2,3,4,5});
```

UNIVERSITY of HULL

Remote Procedure Calling

A remote procedure call is a method call which is invoked outside the currently executing process.

Often this means on another machine altogether…

but it doesn’t have to be.

Even if it is happening on the same machine we still need some way to invoke the function in another piece of software as we can’t just call a method outside of our application.

Remote Procedure Calling

Why would we want to do this?

- Low power clients may need to offload some of their execution to other machines
- If data required for the procedure is not held locally, it may be slow to download all of the data and then run the procedure locally. Instead, just invoke a function on the data server and return the output!
- A server can cache the result of execution so if a common procedure with the same parameters is regularly requested by many clients, the execution only needs to happen once
- If we don't want to expose our code we can ensure critical functions are run on a system we have control over. This can help:
  - Prevent our code from being reverse engineered and stolen or inspected for vulnerabilities
  - Stop unlicenced use of software systems (can't redistribute code you don't have)
  - Reduce the likelihood of viruses or other malicious code affecting operation
  - Ensure the correct resources exist on the machine

UNIVERSITY of HULL

Required Remote Procedure Calling

Some things can only be completed on another machine:

- Calls for a super-computer to run a subroutine on a dataset and return the outcome
  - May take far too long on a desktop machine
- Calls for calculations to be performed that a client doesn't have hardware for
  - E.g. Large-scale graphics calculations on a device with no graphics card capability
- Calls for calculations to be performed which rely on data from other devices which is not all held locally (or can't be verified as locally accurate)
  - E.g. Multiplayer gaming bullet calculations where the function relies on the position of the player firing, the player being shot at and their respective statuses.

The key consideration for Remote Procedure Calling is that we are concerned with a function being executed remotely as if it were being called locally.

UNIVERSITY of HULL

How is this different to SOAP?

SOAP is a mechanism by which we can conduct RPC

For example, a client might use the WSDL contract to know how to make a Remote Procedure Call to a server to execute a remote function. The function's parameters and return value encoded in a SOAP message.

The SOAP message provides the structure and formatting for the Remote Procedure to be called and the data being exchanged.

Proxy Stub

Once the client knows what the remote method name, parameters and return type are, a proxy stub can be created:

This is the equivalent of this:

```
public int Add(int[] integers)
{
    return CallRemoteProcedure(integers);
}
```

In many RPC frameworks (e.g. SOAP), this stub is generated from the contract automatically so we don't have to create it manually. It can be called locally as if the actual work was being done on our machine.

UNIVERSITY of HULL

Calling the Remote Function

```
public int Add(int[] integers)
{
    return CallRemoteProcedure(integers);
}
```

?!

Of course, it's not *quite* that simple.

- The request still needs to be sent from A to B
  - This needs to be done via an application layer protocol of some sort (e.g. HTTP)
- Data encoding needs to be the same on client and server
  - E.g. are we sending the integer array using XML or JSON?
- There may be other requirements in place that need to be fulfilled
  - E.g. encryption of parameters and decryption of the returned value

An RPC protocol gives a definition of how the remote function will be called and how the data will be transferred from one party to the other

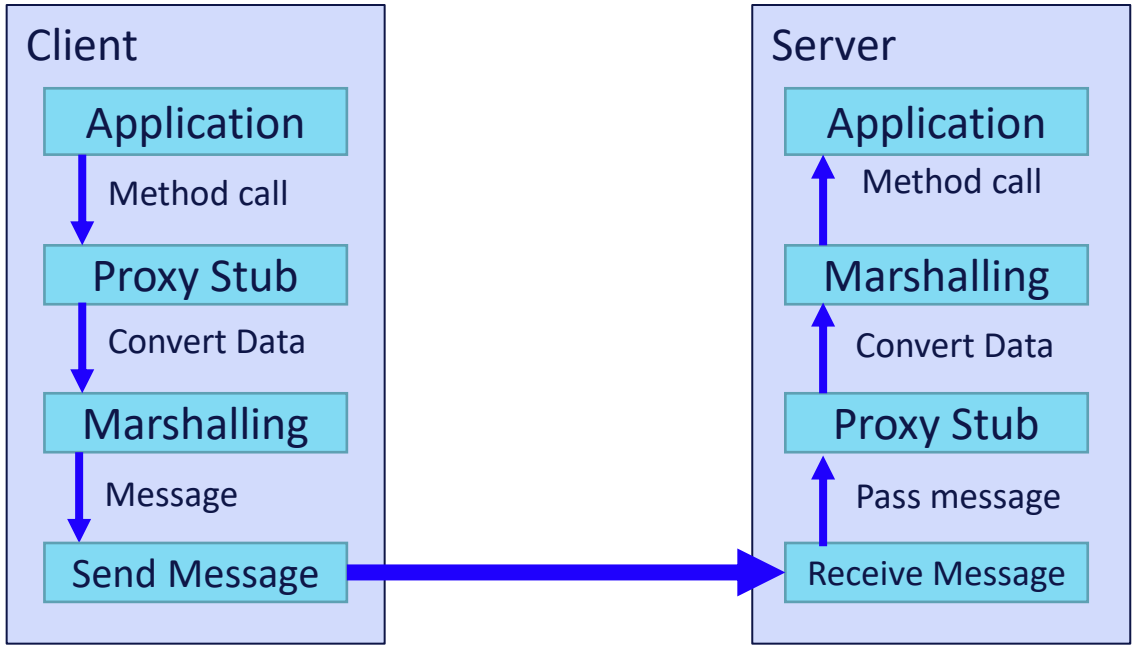
UNIVERSITY of HULL

Marshalling

Data encoding needs to be the same on client and server

- E.g. are we sending the integers using XML or JSON?

Marshalling is the process of converting the local object(s) to the correct format for transmission and then converting the response transmission into local object(s) for the application to use.



UNIVERSITY of HULL

Marshalling (JSON Example)

```
Int[] integers = new int[] {1,2,3,4,5});
Add(integers);
```

Marshalling (client outbound request)

1. **Converts this to JSON:** `{ "integers": [ 1,2,3,4,5 ] }` (this is the message body)
2. **Adds any required headers to the message**
3. **Converts the message to a byte array** (example with no headers shown here in hex):

```
7b 20 22 69 6e 74 65 67 65 72 73 22 3a 20 5b 20 31 2c 32 2c 33 2c 34 2c 35 20
5d 20 7d
```

This is now ready to send to the server's Add endpoint.

The client application usually uses the local operating system to actually send the message via the network.

UNIVERSITY of HULL

Marshalling (JSON Example)

```
Incoming hex:  7b 20 22 72 65 73 75 6c 74 22 3a 20 31 35 20 7d
```

Marshalling (client inbound response)

1. **Converts hex to a string:** `{ "result": 15 }` (this is the message body)
2. **Inspects any necessary headers in the message**
3. **Converts the JSON to an object**: an int32 named *result* with a value of 15

This is passed back as a C# object as the return value for the method call

UNIVERSITY of HULL

How is this different to REST?

This is a good question, as REST can be made to work in much the same way as RPC

Fundamentally there are some core differences:

1. RPC is focussed on calling remote actions whereas REST is primarily concerned with data access and availability

2. REST is based around a URI which identifies where a RESOURCE can be located. RPC is only concerned with actions that can be called, not resources. E.g…
   - www.mydomain.com/users/123/posts <- this is a resource identifier that identifies that we are interested in the posts of the user with the ID 123
   - RPC would instead likely call: www.mydomain.com/getposts and pass the user ID as a parameter to the method

…

UNIVERSITY of HULL

How is this different to REST?

3. REST is focussed on all CRUD functionality and makes use of all of the HTTP action verbs (GET, POST, PUT, DELETE, PATCH, etc.) whereas RPC is typically focussed on GET (call function without parameters) or POST (call function with parameters)

4. Any server functions above CRUD functionality in REST are secondary, functions are the core of RPC – data is secondary.

REST working like RPC

We can make REST work in an RPC-like way and many solutions do just this

E.g.
www.mydomain.com/users/123**/RunFunction**

That's ok but it isn't *technically* a REST architecture.
RunFunction is a procedure, not a resource

Still – lots of distributed applications work in this way…

UNIVERSITY of HULL

REST vs. RPC Understandability

In REST, the resources and what we are doing to them should be clear from the URI and the data

Your users (e.g. other developers) must understand how to interact with the service and what parameters are required and why.

Problems come when things aren't clear:

- When you want to add a user to a group with admin privileges:

We could use:

```
PATCH www.mydomain.com/users/123
{ "group": "superheroes", "role": "admin" }
```

But this doesn't really make sense. The data is probably not stored in the user's account directly, especially as the role is directly associated to the specific group, not to the user

UNIVERSITY of HULL

REST vs. RPC Understandability

RPC can, arguably, be easier to understand:

When you want to add a user to a group with admin privileges:

We could use:

```
POST www.mydomain.com/AddUserToGroupWithRole
{ "userID": 123, "group": "superheroes", "role": "admin" }
```

REST vs. RPC Understandability

Of course there are grey areas…

Is this RPC or REST?

```
POST www.mydomain.com/users/AddUserToGroupWithRole/123
{ "group": "superheroes", "role": "admin" }
```

The lines between RPC and REST are blurred

.NET WebAPI allows you to create a RESTful service functioning more like RPC Although you lose some of the benefits of using an RPC protocol such as:

- Contracts
- Proxy stubs
- Choice of application layer protocol (REST is mapped to HTTP)

UNIVERSITY of HULL

Drawbacks of RPC

- **Often we want client to perform as much of the work as possible**
  - Getting the server to do lots of work often reduces scalability and/or performance
- **Data is what we typically send via the Internet**
  - We don't often actually NEED to focus on remote procedures
- **Getting the contract right is critical for any type of automation**
  - If the contract is wrong, proxy stubs are incorrect and the client and server can't talk to one another. This can make identifying even minor issues difficult.
  - If the contract needs to change, all clients need to be updated and the proxies need to be recreated
  - Contracts need to be exchanged. Usually this needs to happen before you can make use of the remote function (before programming)
- **As it is managed over a network RPC is *extremely slow* compared to local functions. If it doesn't NEED to be done remotely it probably should be done locally.**
  - Unless your client uses async programming it will just have to stop and wait for the remote function to return.

UNIVERSITY of HULL

Common RPC Protocols

Some Examples of RPC Protocols:

- SOAP
- JSON-RPC
- gRPC

Deprecated:

- CORBA
- .NET Remoting

UNIVERSITY of HULL

gRPC

We'll be taking a look at gRPC in the workshops so only core concepts will be covered here.

(What does the g stand for? See: https://grpc.github.io/grpc/core/md_doc_g_stands_for.html)

UNIVERSITY of HULL

Protocol Buffers (protobuf)

Protocol Buffers

- The gRPC interface definition language (IDL) that expresses the **data contract** and allows creation of stub proxies
- Used by both Client and Server for marshalling the data, allowing the transfer of data in a very small and lightweight format

```
service Greeter {
   rpc SayHello (HelloRequest) returns (HelloReply);
}
```

Defines a service called **Greeter** with a function **SayHello**. SayHello requires a **HelloRequest** object as a parameter and returns a **HelloReply** object in response

```
message HelloRequest {
   string name = 1;
   int32 id = 2;
}
```

Defines a message object format containing two variables; a string (name) and an integer (id). These are identified by their field number (which helps with encoding and decoding the messages)

```
message HelloReply {
   string message = 1;
}
```

Defines a message object format containing one variable; a string (message) This also has a field number which must be unique within the message

UNIVERSITY of HULL

Why Field Numbers?

When a message is encoded, the keys and values are concatenated into a byte stream called a **wire-format message**.

When the message is being decoded, the parser needs to be able to skip fields that it doesn't recognize.

- This way, new fields can be added to a message without breaking old programs that do not know about them.

The "key" for each pair in a wire-format message is actually two values

1. the field number from your .proto file,
2. a value that defines the value type (and helps to identify how long the value is)

More detail: https://developers.google.com/protocol-buffers/docs/encoding

Working with Different Clients

Theoretically, you can easily write a library to understand protobuf contracts and convert wire-format messages into local objects in any language.

This allows a server to communicate with different devices with complete language independence.

- The server can be written in C++ and communicate with a C# or Java or Ruby (etc.) client with no problem

Typically, gRPC uses HTTP/2 to communicate

- Conventional REQUEST and RESPONSE format
- Security can be maintained through encryption (HTTPS)
- HTTP is supported on most devices and operating systems

gRPC Limitations

Messages sent using JSON or XML (e.g. SOAP and most REST APIs) are human readable and therefore easier to manually check and use.

- You do not need anything else before you can start using the data.
- You can easily create manual test data

gRPC **requires** the use of a protobuf contract

- You need the protobuf file before the message can be deserialised and understood.
- You also need a functional deserialiser before you can start using the data.
- This means that it can be more difficult for it to be widely shared and used.

Browsers do not all currently support gRPC messages natively

Reflection

In this session we've looked at:

- What is RPC?
- How is RPC different to other architectural styles for building services?
- What are the conventional steps of RPC?
- Benefits and Downsides of RPC
- gRPC

In the next session we will look at Security Essentials

UNIVERSITY of HULL